Graph-Aware Transformers for Smart Contract Vulnerability Detection

Axel Bröns[1], Valentin Kocijancic[1], Antoine Goudedranche[1], Thibault Garel[1], Hugo Rivière[1], Omar El Alami El Fellousse[1]

[1] ECE Ecole d'ingénieurs, 10 Rue Sextius Michel, 75015 Paris

Context & Motivation

- **Stakes** : Smart contracts are immutable and manage billions of dollars. A single vulnerability (e.g., the DAO hack) is irreversible. [2]
- **Current Limitations** : Static analysis tools rely on rigid rules. Recent generative LLMs (e.g., Llama 3.2, DeepSeek) suffer from disastrous accuracy (often below 34%) and excessive latency for auditing purposes.
- **Challenges** : How can we understand both the code's vocabulary (text) and its execution logic (structure) to detect vulnerability?

Experimental Setup

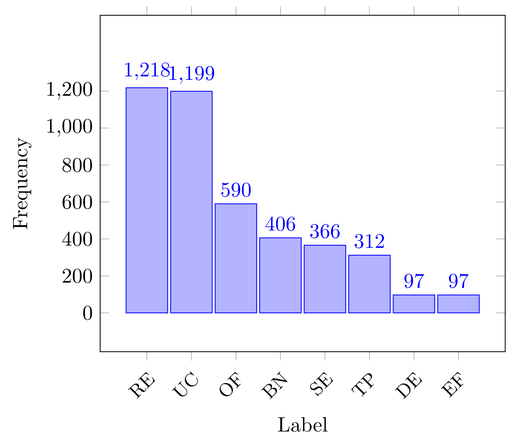


Figure 1. Repartition of the 8 label in the original dataset, where RE is reentrancy, UC is unchecked external call, Of is integer overflow, BN is block number dependency, SE is ether strict equality, TP is timestamp dependency and EF is ether frozen

Setup	RE	UC	OF	BN	SE	TP	DE	EF	Safe
Original	1218	1199	590	406	366	312	97	97	–
9-Class	600	600	590	406	366	312	97	97	1000
7-Class	600	600	590	406	366	312	–	–	1000
Binary	4000 (Aggregated Vulnerable)								4000

Sources:
Vulnerable contracts from Kaggle (8 attack vectors) and safe contracts from verified Etherscan repositories.
Binary Setup:
To strictly eliminate baseline bias, we created a perfectly balanced dataset of 8,000 contracts (50% Safe / 50% Vulnerable).

Proposed Methodology

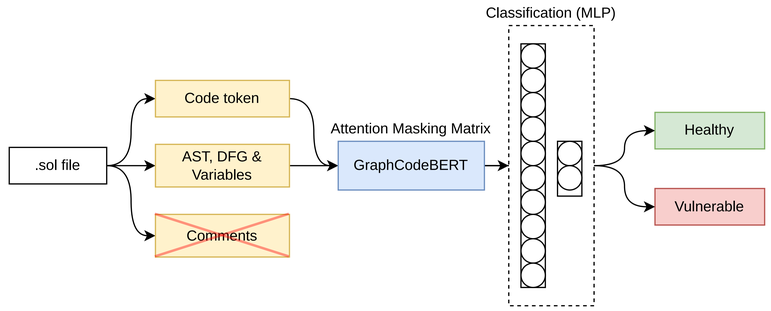


Figure 2. General Pipeline

Beyond Flat Text:
We extract the Abstract Syntax Tree (AST) and the Data Flow Graph (DFG) from raw Solidity code.
Graph-Guided Attention:
GraphCodeBERT [1] uses a specific Attention Masking Matrix that forces the neural network to follow actual data dependencies (where the value comes from), effectively filtering out syntactic noise.
Optimization:
Natural language comments are discarded to maximize the 512-token context window for critical logic.

Graph-Guided Masked Attention:
To explicitly inject the Data Flow Graph (DFG) into the Transformer [3] without altering its architecture, we define a 3D masking matrix M applied directly within the self-attention mechanism:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}} + \widetilde{M}\right)V$$

Where the penalty matrix $\widetilde{M}$ blinds the network to syntactically irrelevant connections:

$$\widetilde{M}_{i,j} = \begin{cases} 0 & \text{if } M_{i,j} = 1 \text{ (Connected in DFG)} \\ -\infty & \text{if } M_{i,j} = 0 \text{ (No data flow)} \end{cases}$$

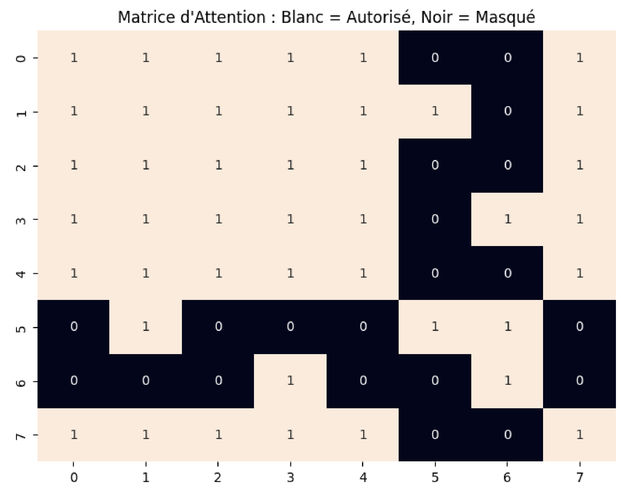
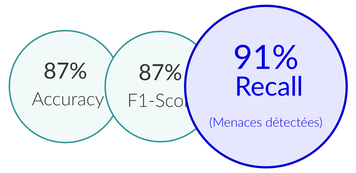


Figure 3. Example of mask attention matrix for the code `x = y`

Key Results

Baseline evaluations revealed that off-the-shelf generative LLMs (e.g., Llama 3.2, DeepSeek-Coder) suffer from severe precision deficits (< 34%) and impractical latencies. Furthermore, even after parameter-efficient fine-tuning (TinyLlama with LoRA), precision remained critically low (22.20%), proving the fundamental inadequacy of flat-text analysis for deterministic security auditing.
Our method:



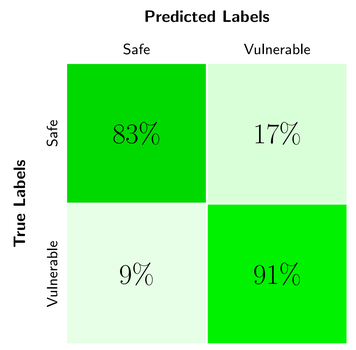


Figure 4. Confusion Matrix for binary classification (544 True Positives, representing a 91% recall)

Conclusion & Perspectives

Conclusion
Integrating structural code logic (DFG) into pre-trained language models offers a highly scalable and robust paradigm for securing decentralized finance, vastly outperforming flat-text analysis.
Future Work

- **Modernization:** Training on modern Solidity compilers (v0.8.x+) to handle built-in arithmetic protections.
- **Scaling:** Implementing sparse attention (e.g., Longformer) to audit massive enterprise-grade contracts without truncating tokens.
- **Zero-Day Detection:** Shifting to unsupervised anomaly detection to flag completely novel exploits.

References

[1] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou.
Graphcodebert: Pre-training code representations with data flow, 2021.

[2] Sarwar Sayeed, Hector Marco-Gisbert, and Tom Caira.
Smart contract: Attacks and protections.
Ieee Access, 8:24416–24427, 2020.

[3] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin.
Attention is all you need.
Advances in neural information processing systems, 30, 2017.